

Unidad 7

Funciones y Procedimientos almacenados

Programación SQL

2

Contenido:

- ▶ Funciones
- ▶ Procedimientos

Programación SQL

3

- ▶ Los **procedimientos almacenados** son elementos de la base de datos que encapsulan varias sentencias SQL con la posibilidad de definir sobre dicha secuencia de sentencias estructuras de control e iteración.
- ▶ Se denominan almacenados porque se almacenan y ejecutan en el servidor, de esta forma se libera a las aplicaciones cliente de accesos y procesos pesados sobre la base de datos.
- ▶ En definitiva, se trata de la programación con SQL para la manipulación de las bases de datos.

- ▶ Los procedimientos (y funciones) almacenados proporcionan:
 - **Compatibilidad:** se definen en el servidor independientemente de la plataforma o lenguaje de programación que utilicen los clientes.
 - **Integridad:** centralizan el acceso a la información
 - **Seguridad:** a través de los permisos de ejecución definidos para cada objeto.
 - **Mejor rendimiento:** El proceso se realiza en el servidor, liberando al cliente del proceso y reduciendo el número de accesos e intercambio de información entre servidor y cliente.
 - **Reutilización del código:** los procedimientos se pueden acceder desde distintas aplicaciones cliente.

- ▶ Otras consideraciones respecto a la definición de rutinas (procedimiento/funciones)
 - Para poder crear procedimientos o funciones es necesario tener el privilegio '**CREATE ROUTINE**'. Los permisos 'ALTER ROUTINE' y EXECUTE se asignan automáticamente al creador.
 - Es posible introducir **comentarios** en el cuerpo de las rutinas con '--'
 - Se trata de un lenguaje '**case insensitive**' esto es, las palabras reservadas se pueden escribir tanto en mayúsculas como en minúsculas. Por defecto también son 'case insensitive' los nombres de tablas, columnas, rutinas, vistas, índices, etc.

- Las rutinas (procedimientos/funciones) se crean de forma predeterminada en la base de datos actual, pero también se puede indicar explícitamente la base de datos en el momento de creación.
- Antes de empezar a definir una rutina hay que **modificar el delimitador** de sentencias ';' por uno diferente, habitualmente // o \$\$\$. Esto es así porque es necesario distinguir el separador de sentencias dentro del cuerpo de la rutina (;) por el del final de la rutina.

Procedimientos

7

- ▶ Los procedimientos se crean con la sentencia **CREATE** y se invocan con **CALL**
- ▶ **Sintaxis:**

```
CREATE  
  [DEFINER = usuario]  
  PROCEDURE nombre_proc* ([parametros[,...]])  
  [características ...] sentencias SQL
```

* *se puede indicar la base de datos: **nombreBD.nombre_proc***

- ▶ **DEFINER:** Cuenta a la que se asigna la rutina. Si no se indica, corresponde al creador.

parametros:

[**IN** | **OUT** | **INOUT**] *nombre_param tipo*

- ▶ Aunque no haya parámetros deben mantenerse los paréntesis: `nom_proc ()`
 - **IN**: para parámetros de entrada al procedimiento (opción por defecto)
 - **OUT**: para parámetros de salida del procedimiento
 - **INOUT**: combinación de los anteriores
 - Los procedimientos pueden utilizar cualquier tipo de los anteriores.


```
caracteristicas: {  
    COMMENT 'string'  
    | LANGUAGE SQL | SQL SECURITY { DEFINER | INVOKER }  
}
```

- COMMENT: Descripción del procedimiento que se muestra con show create procedure/function
- SQL SECURITY tiene dos opciones:
 - DEFINER (por defecto): La rutina se ejecuta con los privilegios de la cuenta indicada como 'DEFINER'.
 - INVOKER: La rutina se ejecuta con los privilegios del usuario que la invoca.

- ▶ Las funciones se crean con **CREATE FUNCTION** y se invocan referenciándolas en una **expresión**. Las funciones **devuelven siempre un valor**

- ▶ **Sintaxis:**

```
CREATE
  [DEFINER = usuario]
  FUNCTION nombre_func* ([parametros[,...]])
  RETURNS tipo
  [características ...] sentencias SQL
```

- ▶ Los **parámetros** de una función siempre son de **tipo IN**, no se indica en la definición.
- ▶ Debe existir al menos un RETURN que devuelva la expresión resultado al invocador. Puede haber más de un return si existen varios puntos posibles de salida de la función.

Alter/Drop/Show

11

- ▶ **ALTER.** Con esta sentencia se pueden modificar las características de un procedimiento o función, pero no se pueden modificar los parámetros o cuerpo de la rutina. Para modificar parámetros o cuerpo, se tiene que eliminar el procedimiento/función y volver a crear la nueva versión.

- ▶ **Sintaxis:**

```
ALTER [PROCEDURE | FUNCTION] nombre_rutina [caracteristicas ...]
```

```
caracteristicas: {  
    COMMENT 'string'  
    | LANGUAGE SQL  
    | SQL SECURITY { DEFINER | INVOKER }  
}
```

Alter/Drop/Show

12

- ▶ **DROP.** Para eliminar el objeto procedimiento o función.

- ▶ **Sintaxis:**

- ```
DROP {PROCEDURE | FUNCTION} [IF EXISTS] nombre_rutina
```

- ▶ **SHOW:** Con las sentencias SHOW se puede ver el detalle de la creación del objeto.

- ▶ SHOW CREATE PROCEDURE miproc;

- ▶ En **MySQL** se pueden declarar variables de diferentes formas:
  1. Variables de sesión
  2. Variables locales en procedimientos o funciones
  3. Variables en una consulta

## ► Variables de sesión

- No requieren declaración previa
- Se identifican con una @ delante del nombre
- Persisten durante la sesión de la conexión.
- Se utiliza **SET** para asignar un valor
- Se utiliza **SELECT** para consultar el valor
- Ejemplo

SET @miVar = 7;

SELECT @miVar;

## ► Variables locales en procedimientos o funciones

- Solo pueden utilizarse en procedimientos o funciones
- Deben declararse al inicio del bloque BEGIN
- No llevan @ en el nombre
- Se introducen con la palabra reservada **DECLARE**

### ► Ejemplo

```
DELIMITER //
CREATE PROCEDURE miProc ()
BEGIN
 DECLARE mi_var INT; -- Declaración
 SET mi_var = 10; -- Inicialización
 SELECT mi_var; -- Uso de la variable
END //
DELIMITER ;
CALL MiProcedimiento();
```

## ► Variables en una consulta

- Almacenan el resultado de una consulta
- Pueden utilizarse con variables de sesión (@) o locales (declare en proc/func)
- Debe utilizarse cuando la SELECT devuelve un **único registro**
- Ejemplo:

```
SELECT nomeq, director
INTO v_nomeq, v_direc
FROM equipo
WHERE equipo= 'Banesto';
```



# Estructura procedimientos/funciones

17

- **BEGIN... END**

- el bloque de sentencias que se ejecutará se incluye entre BEGIN y END.
- Cada sentencia dentro del bloque finaliza con un punto y coma ;

- **DECLARE:** abre la sección de definición de distintos elementos en el ámbito local a una rutina : variables, condiciones, handlers (manejadores) y cursores. Si se utiliza debe incluirse justo a continuación del BEGIN.

- Declaración de variables

DECLARE nombre\_variable [, ...] TYPE tipo\_variable [DEFAULT valor\_por\_defecto]

- **SET:** establece el valor para variables locales o globales (del servidor).

SET nombre\_variable = expr [,nombre\_variable2 = expr2...]